

Git Flow, Conventional Commits y Versionado Semantico

Guia completa para un flujo de trabajo profesional en equipo
Lectura completa: ~60 minutos
Parte 1 — Conventional Commits y Validacion

1. Conventional Commits

Conventional Commits es una especificacion ligera para mensajes de commit que permite:

- Generar **CHANGELOGs automaticamente**
- Determinar cambios de **version semantica automaticamente**
- Mejorar la **comunicacion** entre desarrolladores, DevOps y herramientas automatizadas

1.1 Formato del mensaje

```
<tipo>[<ambito>][!]: <asunto>

[cuerpo]

[BREAKING CHANGE: <descripcion>]

[notas de pie]
```

Regla de commit que determina el tipo de cambio

```
# Para un FIX (patch): commit con tipo fix
fix: corregir calculo de premios en sorteo

# Para un FEATURE (minor): commit con tipo feat
feat: agregar exportacion de resultados a PDF

# Para un BREAKING CHANGE (major): !
refactor!(auth): cambiar de RSA a JWT

# 0 con BREAKING CHANGE en el pie de commit:
feat(api): cambiar estructura de respuesta

BREAKING CHANGE: Campo 'data' ahora es un objeto
```

1.2 Tipos principales

Tipo	Descripcion	Version SemVer
feat	Nueva funcionalidad visible para el usuario	MINOR (1.0.0 → 1.1.0)
fix	Correccion de un bug (error conocido)	PATCH (1.0.0 → 1.0.1)
BREAKING CHANGE	Cualquier cambio que rompa compatibilidad hacia atras	MAJOR (1.0.0 → 2.0.0)

1.3 Tipos adicionales

Tipo	Cuando usar	Ejemplo
build	Sistema de build o dependencias	build: upgrade gradle a 8.0
chore	Trabajo de mantenimiento	chore: limpiar cache de dependencias
ci	Integracion continua	ci: agregar step de codegen en CI
docs	Documentacion	docs: actualizar guia de instalacion
perf	Mejora de rendimiento	perf: cachear consultas de base de datos
refactor	Refactorizacion (sin cambio de funcionalidad)	refactor: extraer widget comun
style	Cambios de formato	style: dart format en todo el proyecto
test	Agregar o corregir tests	test: cubrir caso borde en sorteo

1.4 Reglas de los asuntos (subjects)

- Usar **imperativo presente**: "agregar feature", no "agregue feature" ni "haciendo feature"
- **Sin punto final**
- Maximo **72 caracteres**
- **Primera letra en minusculas**

1.5 El ambito (scope)

Opcional, agrega contexto adicional:

```
feat(raffles): agregar filtro por fecha
fix(auth): corregir login con Google
feat(export): agregar exportacion PDF
```

1.6 Breaking Changes

```
# Opcion 1: ! despues del tipo
refactor!(auth): cambiar respuesta de login a JWT
```

```
# Opcion 2: BREAKING CHANGE en el pie
feat: migrar de SQLite a Isar
```

```
BREAKING CHANGE: Se elimino la base de datos local. Los usuarios deben exportar sus datos antes de actualizar.
```

1.7 Notas al pie

```
feat: agregar autenticacion con Google
```

```
Closes #123, Fixes #456, Refs #789
```

1.8 Lenguaje natural

Se pueden agregar colas de texto despues de los prefijos:

```
feat: agregar autenticacion con Google y Facebook
```

1.9 COMMIT_MESSAGES.md para el proyecto

Crear un archivo COMMIT_MESSAGES.md en la raiz del proyecto para estandarizar:

```
# Conventional Commits para [Nombre del Proyecto]

## Tipos permitidos
- feat: Nueva funcionalidad
- fix: Correccion de bug
- refactor: Refactorizacion sin cambio funcional
- docs: Documentacion
- test: Tests
- chore: Mantenimiento
- build: Build system
- ci: CI/CD
- perf: Rendimiento
- style: Formato

## Ambitos comunes
- auth: Autenticacion
- raffles: Sorteos
- export: Exportacion
- ui: Interfaz de usuario

## Ejemplo
feat(raffles): agregar filtro por fecha en lista de sorteos
```

1.10 Relacion con SemVer

Conventional Commits permite calcular automaticamente la version SemVer:

```
# Version actual: 1.0.0

# Si el ultimo commit es:
fix: corregir calculo → 1.0.1 (PATCH)
feat: agregar filtro → 1.1.0 (MINOR)
feat!: cambiar API → 2.0.0 (MAJOR)

# Con standard-version, esto es automatico
standard-version --first-release
```

2. Husky + lint-staged

Husky ejecuta scripts automaticos en hooks de Git. **lint-staged** ejecuta comandos SOLO en archivos staged (preparados para commit).

2.1 Instalacion

```
# Husky (moderno, v9+)
npm install --save-dev husky
npx husky init
# Crea .husky/ con hooks de ejemplo

# lint-staged
npm install --save-dev lint-staged
```

2.2 Configuración en package.json

```
{
  "scripts": {
    "prepare": "husky"
  },
  "lint-staged": {
    "*.dart": [
      "dart format --set-exit-if-changed",
      "flutter analyze --fatal-infos"
    ],
    "*.{yaml,json}": [
      "prettier --write"
    ]
  }
}
```

2.3 Los 3 hooks principales

Hook	Cuando se ejecuta	Que hace	Importancia
pre-commit	Antes del commit	Ejecuta lint-staged (análisis + formato)	CALIDAD
commit-msg	Después del mensaje	Valida formato (commitlint)	CONVENCION
pre-push	Antes del push	Ejecuta tests completos	FIABILIDAD

2.4 Configuración del pre-commit

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx lint-staged
```

2.5 Configuración del commit-msg

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx --no -- commitlint --edit ${1}
```

2.6 Configuración del pre-push

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

flutter test
```

2.7 Orden de ejecucion

```
$ git commit -m "feat: agregar filtro"

# 1. Husky ejecuta pre-commit
→ dart format --set-exit-if-changed
→ flutter analyze --fatal-infos

# 2. Husky ejecuta commit-msg
→ commitlint --edit .git/COMMIT_EDITMSG

# 3. Commit exitoso ✓
```

2.8 Configuración avanzada de lint-staged

```
{
  "lint-staged": {
    "*.dart": {
      "dart format --set-exit-if-changed": true,
      "flutter analyze --fatal-infos": true
    },
    "*.yaml": ["yamllint"],
    "*.json": ["prettier --write"],
    "*.md": ["prettier --write"]
  }
}
```

2.9 Depuración de Husky

```
# Verificar que Husky está instalado
ls .husky/

# Verificar los hooks
cat .husky/pre-commit
cat .husky/commit-msg

# Ejecutar manualmente un hook
npx lint-staged
```

2.10 Emergencias: saltarse los hooks

```
# Saltar todos los hooks (no recomendado)
git commit --no-verify -m "fix: hotfix critico"

# Forzar commit sin validación
git commit -am "fix: fix urgente" --no-verify
```

2.11 Consideraciones para CI/CD

```
# En GitHub Actions, Husky NO se ejecuta automaticamente
# Los hooks solo corren en el entorno local

# Solucion: Ejecutar las mismas validaciones en CI
- name: Validate commits
  run: npx commitlint --from ${ github.event.pull_request.base.sha } --to ${ github.event.pull_request.head.sha }

- name: Lint staged
  run: npx lint-staged
```

2.12 Reconocimiento de committer con .mailmap

```
# Archivo .mailmap en la raiz del repo
# Une multiples identidades de email/usuario

Nombre Real <email_real@example.com> <email_anterion@example.com>
```

3. Commitizen + commitlint

Commitizen = CLI interactivo que genera commits con formato correcto. **commitlint** = linter que valida mensajes de commit contra las reglas.

3.1 Instalacion

```
# Commitizen
npm install --save-dev commitizen cz-conventional-changelog

# commitlint
npm install --save-dev @commitlint/cli @commitlint/config-conventional
```

3.2 Configuracion en package.json

```
{
  "scripts": {
    "commit": "cz",
    "prepare": "husky"
  },
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

3.3 Flujo con Commitizen

```
$ npm run commit
# o
$ npx cz

? Select the type of change:
  feat
? Scope (optional):
  raffles
? Short description:
  agregar filtro por fecha
? Breaking changes (optional):
  No

# Resultado:
feat(raffles): agregar filtro por fecha
```

3.4 Configuración de commitlint

```
// commitlint.config.js
module.exports = {
  extends: ['@commitlint/config-conventional'],
  rules: {
    'type-enum': [2, 'always', [
      'feat', 'fix', 'docs', 'style', 'refactor',
      'perf', 'test', 'build', 'ci', 'chore', 'revert'
    ]],
    'type-empty': [2, 'never'],
    'subject-empty': [2, 'never'],
    'subject-full-stop': [2, 'never', '.'],
    'header-max-length': [2, 'always', 72],
    'body-leading-blank': [2, 'always'],
    'footer-leading-blank': [2, 'always']
  }
};
```

3.5 Reglas de commitlint

Regla	Valor	Descripción
type-enum	[2, 'always', [...]]	Tipos permitidos en commits
type-empty	[2, 'never']	El tipo es obligatorio
subject-empty	[2, 'never']	La descripción es obligatoria
header-max-length	[2, 'always', 72]	Máximo 72 caracteres en la primera línea
subject-full-stop	[2, 'never', '.']	No usar punto final en la descripción

3.6 Estandarizar con COMMIT_MESSAGES.md

```
feat(raffles): agregar filtro por fecha
fix(auth): corregir login con Google
feat(export): agregar exportacion PDF
refactor(ui): extraer widget de sorteo
chore(deps): actualizar dependencias
test(raffles): cubrir caso borde en sorteo
docs(api): documentar endpoint de login
```

3.7 Standard Version: CHANGELOG automatico

```
# Instalar
npm install --save-dev standard-version

# Configurar en package.json
{
  "standard-version": {
    "header": "# CHANGELOG",
    "types": [
      {"type": "feat", "section": "Features"},
      {"type": "fix", "section": "Bug Fixes"},
      {"type": "perf", "section": "Performance"},
      {"type": "refactor", "section": "Refactor"},
      {"type": "docs", "section": "Documentation"}
    ]
  }
}

# Usar
npm run release

# Esto genera:
# - CHANGELOG.md actualizado
# - package.json con version bump
# - Commit con los cambios
# - Tag v1.2.0
```

3.8 Generacion de CHANGELOG con git-cliff

```
# Instalar
cargo install git-cliff

# Ejecutar
cliff -o CHANGELOG.md

# Resultado automatico desde los commits
# basado en Conventional Commits
```

3.9 Integracion completa (Husky + Commitizen + commitlint)


```
# Flujo completo para el desarrollador:
1. npx cz # Genera commit correcto
2. Husky ejecuta pre-commit # Valida formato
3. Husky ejecuta commit-msg # Valida con commitlint
4. git push # Push a origin
5. Pre-push ejecuta tests

# En CI/CD:
1. CI ejecuta commitlint # Valida todos los commits del PR
2. CI ejecuta lint-staged # Valida archivos modificados
3. CI ejecuta flutter test # Valida que no se rompa nada
```

Parte 2 — Versiones, Ramas y Releases

4. FVM: Flutter Version Management

FVM permite gestionar multiples versiones de Flutter en el mismo equipo. Cada proyecto tiene su propia version.

4.1 Instalacion

```
# Instalar FVM
dart pub global activate fvm

# Instalar Flutter 3.24.0
fvm install 3.24.0

# Usar en el proyecto
cd mi_proyecto
fvm use 3.24.0
```

4.2 Comandos esenciales

Comando	Que hace
fvm install <version>	Instala una version de Flutter
fvm use <version>	Configura la version del proyecto actual
fvm flutter pub get	Ejecuta pub get con la version configurada
fvm flutter run	Ejecuta el proyecto con la version configurada
fvm flutter test	Ejecuta tests con la version configurada
fvm list	Lista todas las versiones instaladas
fvm current	Muestra la version actual del proyecto
fvm global <version>	Establece version global del sistema

4.3 Archivos importantes

```
# Archivo de version (SE COMITA)
.fvm/flutter_sdk_version

# SDK instalado (SE IGNORA en .gitignore)
.fvm/flutter_sdk
```

4.4 .gitignore

```
# Agregar a .gitignore
.fvm/flutter_sdk
.fvm/fvm_config.json
```

4.5 Configuración VS Code

```
// .vscode/settings.json
{
  "dart.flutterSdkPath": ".fvm/flutter_sdk",
  "dart.flutterBinaryPath": ".fvm/flutter_sdk/bin/flutter"
}
```

4.6 CI/CD con FVM

```
# GitHub Actions
- name: Setup Flutter
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.24.0'

# 0 con FVM
- name: Setup FVM
  run: |
    dart pub global activate fvm
    fvm install
    fvm flutter pub get

- name: Run tests
  run: fvm flutter test
```

4.7 Docker con FVM

```
# Dockerfile
FROM ghcr.io/cirruslabs/flutter:3.24.0

WORKDIR /app
COPY . .
RUN flutter pub get
RUN flutter build apk
```

4.8 Comparativa de herramientas

Herramienta	Ventajas	Desventajas
-------------	----------	-------------

FVM	Rapido, simple, IDE integration	Requiere configuracion inicial
asdf	Multi-lenguaje, plugin system	Mas complejo, overhead
Flutter solo	Sin dependencias extra	Solo una version global

5. Estrategias de Ramas y Releases

Define como organizar el trabajo en equipo: que ramas existen, como se integran, y como se hacen los releases.

5.1 Conventional Branch

Conventional Branch es una especificacion para agregar significado legible a las ramas Git.

```
# Formato general
<tipo>/<descripcion>
```

Prefijos estandar

Prefijo	Proposito	Ejemplo
main / master	Rama principal de desarrollo	main
feature/ o feat/	Nuevas funcionalidades	feature/agregar-login
bugfix/ o fix/	Corregir errores	bugfix/corregir-crash
hotfix/	Correcciones urgentes	hotfix/parche-seguridad
release/	Preparar una liberacion	release/v1.2.0
chore/	Tareas sin codigo	chore/actualizar-deps

Reglas de nomenclatura

- Alfanumericos, guiones y puntos en minusculas
- Sin guiones o puntos consecutivos
- Sin guiones/puntos al inicio o final
- Claro y conciso
- Incluir numeros de ticket si aplica

5.2 Git Flow vs GitHub Flow

Aspecto	Git Flow	GitHub Flow
Ramas	main, develop, feature/*, release/*, hotfix/*	main, feature/*
Complejidad	Alta	Baja
Releases	Programados	Continuos
Equipo	5+ personas	1-3 personas
Proyecto	App con versiones semanticas	Web/API con deploys continuos

5.6 Versionado Semantico (SemVer 2.0.0)

```
v<major>.<minor>.<patch>

v1.0.0  → Primer release estable
v1.1.0  → Nueva feature
v1.1.1  → Bug fix
v2.0.0  → Breaking change
```

Reglas de incremento

Version	Cuando incrementar	Ejemplo
Major (X)	Cambios incompatibles con API anterior	1.0.0 → 2.0.0
Minor (Y)	Nueva funcionalidad compatible	1.0.0 → 1.1.0
Patch (Z)	Correccion de bug compatible	1.1.0 → 1.1.1

Versiones de pre-lanzamiento

```
v1.0.0-alpha
v1.0.0-alpha.1
v1.0.0-beta.1
v1.0.0-rc.1
```

5.7 CHANGELOG

```
# Changelog

## [1.2.0] - 2026-06-15

### Added
- Exportacion de resultados a PDF
- Notificaciones push por sorteo

### Fixed
- Crash al abrir sorteo sin conexion
- Calculo incorrecto de premios

### Changed
- Migracion de SQLite a Isar
```

5.8 Buenas practicas

- **Commits atomicos:** Cada commit es una unidad logica independiente
- **Ramas cortas:** Maximo 2-3 dias de trabajo, PRs < 400 lineas
- **Rebase en feature:** Historia lineal en ramas de feature
- **Merge en compartidas:** Preservar contexto en ramas compartidas
- **main estable:** Solo mergea desde release/ o hotfix/
- **Tags siempre:** Taggear cada merge a main